2026-08-21-bob131-container-death-triage

BOB-131 — “qbittorrent-proxy podman conmon crash”: two different events, neither a conmon crash

Revision: 1 **Last modified:** 2026-08-21T17:06:26Z **Tracker:** BOB-131 (Bug, Queued, Medium) **Verdict (§11.4.6): The ticket’s premise is FALSE as written.** No `conmon` process crashed. BOB-131 conflates **two unrelated events**, each now identified from primary evidence: 1. **2026-08-19 22:41:39 CEST — a host power-off.** The container exited **0** (clean SIGTERM) and was removed with every other container on the machine. It was then **absent for 14h 06m 43s** because the boba stack has **no boot-persistence unit enabled**. Class: `HOST-POWER-TRANSITION`. 2. **2026-08-20 17:56:26 CEST — a genuine SIGSEGV inside the container.** `died exit=139` (128+11). The crash is in **CPython’s own traceback dumper**, reached from a **project-owned diagnostic** (the BOB-137 stall watchdog calling `faulthandler.dump_traceback(all_threads=True)`), and it is a **known upstream CPython defect fixed in 3.13/3.14 but not in the 3.12 the image ships**. Class: `PROCESS-SIGSEGV`.

Answer to the task’s a/b/c: (a) a real project-side trigger of (b) a known runtime defect. It is *latent, not currently firing*, and it is **not reproducible without deliberately stalling the event loop** — see Reproduction. No fix was manufactured; the code that must change is owned by BOB-137 and is outside this investigation’s file scope.

---

1. The real error text

The ticket says “podman common crash”. The primary record says this (`journalctl`, host TZ **Europe/Belgrade CEST = UTC+02:00**):

```
Aug 20 17:56:26 nezha kernel: python3[314359]: segfault at 70 ip
00007fb2d690fea0 sp 00007fb2d58d2220 error 4 in
libpython3.12.so.1.0[141ea0,7fb2d68d2000+2c3000] likely on CPU 1
(core 1, socket 0)
Aug 20 17:56:26 nezha kernel: Code: 24 46 03 0f 84 d1 00 00 00 41
83 ef 01 0f 84 2d 01 00 00 4d 8b 34 24 ba 07 00 00 00 48 8d 35 ea
14 35 00 89 df e8 10 33 fc ff <49> 8b 46 70 48 85 c0 74 11 48 8b
40 08 f6 80 ab 00 00 00 10 0f 85
Aug 20 17:56:26 nezha kernel: audit: type=1701
audit(1787241386.129:755): auid=1000 uid=1000 gid=1000 ses=1
pid=314359 comm="python3" exe="/usr/local/bin/python3.12" sig=11
res=1
Aug 20 17:56:26 nezha audit[314359]: ANOM_ABEND auid=1000 uid=1000
gid=1000 ses=1 pid=314359 comm="python3" exe="/usr/local/bin/
python3.12" sig=11 res=1
```

and the container’s own stdout, captured because podman’s log driver is `journald`:

```
2026-08-20T17:56:26+02:00 nezha qbittorrent-proxy[314357]: Thread
0x00007fb2d551eb38 (most recent call first):
2026-08-20T17:56:26+02:00 nezha qbittorrent-proxy[314357]:   File
"/usr/local/lib/python3.12/enum.py", line 1294 in value
2026-08-20T17:56:26+02:00 nezha qbittorrent-proxy[314357]:   File
"/usr/local/lib/python3.12/enum.py", line 212 in __get__
2026-08-20T17:56:26+02:00 nezha qbittorrent-proxy[314357]:   File
Fatal Python error: Segmentation fault
2026-08-20T17:56:26+02:00 nezha qbittorrent-proxy[314357]:
Extension modules: multidict._multidict, yarl._quoting_c,
propcache._helpers_c, aiohttp._http_writer, aiohttp._http_parser,
aiohttp._websocket.mask, aiohttp._websocket.reader_c,
frozenlist._frozenlist, rapidfuzz._feature_detector_cpp,
rapidfuzz.distance._initialize_cpp,
rapidfuzz.distance.metrics_cpp_avx2, rapidfuzz.fuzz_cpp_avx2,
rapidfuzz.process_cpp_impl, rapidfuzz.utils_cpp,
Levenshtein.levenshtein_cpp, _cffi_backend (total: 16)
```

Read the third-from-last line carefully: **`File Fatal Python error:`**. The dumper wrote the literal `"  File "` for the next frame and then died mid-line; the SIGSEGV handler’s banner was spliced into the same stream. That truncation is the signature, and it matches the machine code exactly (§3).

No conmon crash appears anywhere. Over the full journal retention (2026-08-14T05:20:28+02:00 → now) the only `conmon` messages above `<nwarn>` are four, none of them a conmon fault:

```
conmon <CID> <error>: Failed to create container: exit status
1                  (x2)
conmon <CID> <error>: Failed to write 137 to exit file: ... No
such file or directory  (x2)
```

Those are conmon **reporting** a condition. A conmon crash would put `conmon` itself in a kernel `segfault`/`ANOM_ABEND` line; the only such line in 7.5 days is the `python3` one above. Treating "a `conmon <error>:` string in the log" as "conmon crashed" is a §11.4.201 carrier match — the token mentions the thing, it is not the thing.

---

2. When it last occurred, and whether it still does

Occurrences of this SIGSEGV in the whole journal retention	**exactly 1**
Last (and only) occurrence	**2026-08-20 17:56:26 CEST** = `2026-08-20T15:56:26Z`
Occurrences since	**0** — zero `Fatal Python error`, zero stall dumps since `2026-08-20T15:56:27`
Container now	`Up`, `healthy`, `RestartCount=0`, `ExitCode=0`, CID `979cbb757b77…`
Is the mechanism still armed?	**YES** — see §5

It recovered by itself: podman's `restart: unless-stopped` policy restarted the container **0.03 s** later.

```
Aug 20 17:56:26.231718 container died     0dcf880eeb6e…   (exit
139)
Aug 20 17:56:26.265280 container restart  0dcf880eeb6e…
Aug 20 17:56:26.376375 container init     0dcf880eeb6e…
Aug 20 17:56:26.386894 container start    0dcf880eeb6e…
```

conmon did its job perfectly: it observed the SIGSEGV, reported exit 139, and the restart policy fired. **The runtime behaved correctly throughout.**

---

3. The conditions — what actually triggered it

Not a restart, not a teardown, not an exec. It was a **project-owned diagnostic running on a live 18-thread process**.

Four seconds of context, from the container's own stdout:

```
2026-08-20T17:50:39+02:00 qbittorrent-proxy[314357]: ===== BOB-137
STALL DUMP #12 episode=2 utc=2026-08-20T15:50:39Z
loop_silent_for=82.0s loop_tid=11 =====
```

That banner is emitted by `download-proxy/src/main.py`:

```
sink.write(f"\n===== BOB-137 STALL DUMP #{_diag_dumps}
        episode={episode} ...")
faulthandler.dump_traceback(file=sink, all_threads=True)     #
        main.py:135
```

The BOB-137 stall watchdog fires when the asyncio event loop goes silent for `>20 s` and re-dumps every 60 s. On 2026-08-20 it fired **17 times** in 16 minutes (`download-proxy/diagnostics/stall_dumps.log`, `#1` at `15:40:04Z` through `#17` at `15:56:17Z`). Dump `#17` completed to the **file** sink, then the **stderr** pass of the same dump crashed 9 seconds later at `15:56:26Z`.

The instrumentation's own source comment names the exact trade-off that killed it:

> `faulthandler.dump_traceback()` is implemented in C and does not need the GIL, so it can still dump when a Python thread is hogging it – which is exactly why `sys._current_frames()` was not used here (it needs the GIL and would hang in the same way the thing it is measuring does).

Because it does not take the GIL, it walks other threads' frame chains while those threads are mutating them. That is the race.

3.1 Machine-level proof it is that exact code path

The kernel's `Code:` dump was byte-matched against the library on disk inside the running container (pure-Python ELF read, no binutils in the image):

```
kernel Code: bytes at <IP>: 49 8b 46 70 48 85 c0 74 11 48 8b 40 08
f6 80 ab 00 00 00 10 0f 85
library bytes at 0x141ea0  : 49 8b 46 70 48 85 c0 74 11 48 8b 40
08 f6 80 ab 00 00 00 10 0f 85
MATCH: True
kernel Code: bytes before <IP>: 4d 8b 34 24 ba 07 00 00 00 48 8d
35 ea 14 35 00 89 df e8 10 33 fc ff
```

```
library bytes before 0x141ea0 : 4d 8b 34 24 ba 07 00 00 00 48 8d
35 ea 14 35 00 89 df e8 10 33 fc ff
MATCH: True
```

Decoded, that is CPython's `dump_frame()`:

```
4d 8b 34 24              mov  r14, [r12]          ; code = frame-
        >f_executable   -> NULL
ba 07 00 00 00           mov  edx,
        7               ; strlen("  File ")
48 8d 35 ea 14 35 00     lea  rsi, [rip+0x3514ea] ; -> the string
        "  File "
89 df                    mov  edi, ebx            ; fd
e8 10 33 fc ff           call
        <_Py_write_noraise> ; writes "  File "   <-- the truncated
        log line
49 8b 46 70          <-- mov  rax, [r14+0x70]     ; code-
        >co_filename   *** FAULT: r14 == NULL ***
48 85 c0                 test rax, rax            ; if
        (co_filename != NULL
74 11                    je   ...
48 8b 40 08              mov  rax, [rax+8]        ;     &&
        Py_TYPE(co_filename)
f6 80 ab 00 00 00 10     test byte [rax+0xab], 16  ;     tp_flags
        & Py_TPFLAGS_UNICODE_SUBCLASS
```

- The `lea` operand resolves to the literal `"  File "` with `edx = 7` — its exact length — which is precisely the text the log line was truncated after.
- The faulting address is `0x70` (NULL + `offsetof(PyCodeObject, co_filename)`).
- Nearest preceding exported symbol is **`_Py_DumpDecimal`**, a `Python/traceback.c` function, `+0x200` before the fault.

The faulting binary is container-side, not host-side: `/usr/local/bin/python3.12` **does not exist on this host** (host Python is `/usr/bin/python3`, **3.14.6**); inside the container it is Python **3.12.13**, image `docker.io/library/python:3.12-alpine`, digest `sha256:aa679aa4eed6eb56c1dc6ad3f1b98b7d2d788fd961596779d188fdedad97fb38`.

3.2 Upstream — this is a known CPython bug (§11.4.99, fetched 2026-08-21)

- **python/cpython#116008** — "Segfault in faulthandler signal handler with threads". Crashes in **`dump_frame()` at `Python/traceback.c:1190` accessing `code->co_filename`** — the same function and the same field. Cause: faulthandler touching thread state / frame objects that other threads are concurrently changing. **Status: closed/fixed**, backports listed for **3.13 (#142017)** and **3.14 (#142013)**.
- **python/cpython#128400** — crash in `_Py_DumpTracebackThreads` where **`_PyFrame_GetCode()` finds `executable` NULL** — the same

NULL `f_executable` this crash loaded. Fix direction: *"Stop-the-world when manually calling `faulthandler`"*.

No 3.12 backport is listed in either issue. The image ships 3.12.13, so staying on `python:3.12-alpine` does not acquire the fix. (§11.4.6: I did not verify 3.12's branch policy directly — recorded as *not listed*, not as *refused*.)

---

4. The other half of the ticket — the 14-hour absence

This is the event the BOB-129 subagent actually walked into, and it is **not a crash at all**.

```
Aug 19 22:41:39 nezha systemd-logind[1296]: The system will power
off now!
Aug 19 22:41:39 nezha systemd-logind[1296]: System is powering
down.
Aug 19 22:41:39 nezha systemd[1]: Stopped target graphical.target
- Graphical Interface.
Aug 19 22:41:39 nezha systemd[1]: Stopping gdm.service - GNOME
Display Manager...
Aug 19 22:41:39 nezha qbittorrent-proxy[1168513]: 2026-08-19
20:41:39,920 - INFO - Shutting down...
```

`systemd[1]` — the **system** manager, not the user manager — stopping `graphical.target`, `gdm`, `getty`, `bluetooth`, `timers`. The container ran its own SIGTERM handler and exited **0**. Boot `-3` ended `22:42:02 CEST`.

Answer to BOB-131's question (a) "how long was the container dead before discovery":

Event	Timestamp (UTC)
`died exit=0` → `cleanup` → `remove`	`2026-08-19T20:41:54Z`
`create` (recovery via `./start.sh`)	`2026-08-20T10:48:37Z`
Absent	**14h 06m 43s**

The host itself was back up at `2026-08-19 23:35:30 CEST` (boot `-1`), so for **13h 13m of that window the machine was running with the boba stack simply not started.**

Why it did not come back on its own

`restart: unless-stopped` is a *within-podman* policy. It does not survive a host power cycle. Boot persistence needs a unit, and **none is enabled**:

```
podman-restart.service           disabled   (user scope)
podman-restart.service           disabled   (system scope)
boba-stack.service               linked     disabled
boba-webui-bridge.service        linked     disabled
loginctl show-user milosvasic -p Linger  ->  Linger=yes
```

`boba-stack.service` **exists and is linked but is not enabled**, and lingering is already on — so the capability is present and one `systemctl --user enable` away. **That is an operator decision and touching it is outside this investigation's file scope; it is handed over in §6, not performed.**

---

5. Is the crash mechanism still armed? Yes.

```
2026-08-21T17:20:39+02:00 qbittorrent-proxy[2449341]: BOB-137
stall watchdog armed: stall>20.0s, redump every 60.0s,
SIGUSR1=dump-to-log SIGQUIT=dump-to-file
```

Defaults apply (`BOBA_STALL_WATCHDOG` unset in the container ⇒ on; `_DIAG_STALL_S=20`, `_DIAG_REDUMP_S=60`, `_DIAG_MAX_DUMPS=200`).

It has not fired since the crash because BOB-137's root cause — dedup running `O(N²)` regex work synchronously on the asyncio event loop, fixed in `1dd7b0a` — no longer stalls the loop past 20 s. **The dumper is still wired to the same unsafe API.** Any future ≥20 s loop stall re-enters the identical race.

This is the honest statement of residual risk: **latent, armed, one trigger away, and the trigger is exactly the condition the watchdog exists to observe.**

---

6. Handed to the operator — NOT performed

Both items below were deliberately not executed: the stack is live, three other agents are working, and neither is mine to decide.

(i) Reproduction. Requires deliberately stalling the event loop on a running container. Do this in a throwaway stack, never the operator's:

```
# NOT run here. Requires a disposable stack; will crash the proxy
        container.
./start.sh                    # a scratch checkout, not the live
        one
podman exec qbittorrent-proxy python3 -c '
```

```
import faulthandler, threading, time
def spin():
    while True:                      # churn frames so
        f_executable is
        (lambda: (lambda: None)())() # transiently NULL for a
        walker
threading.Thread(target=spin, daemon=True).start()
for _ in range(10000):
    faulthandler.dump_traceback(all_threads=True)   # racy by
        construction
'
# Expected: exit 139 + a kernel ANOM_ABEND sig=11 for /usr/local/
        bin/python3.12.
# §11.4.6: this reproduction is CONSTRUCTED, not traced from the
        incident's own
# preconditions, so per §11.4.115(G) it can mint DEFENSIVE
        HARDENING only --
# it cannot close BOB-131 as "fixed".
```

(ii) Boot persistence (closes the 14h-absence half):

```
systemctl --user enable --now boba-stack.service     # operator
        decision
# or:  systemctl --user enable podman-restart.service
systemctl --user is-enabled boba-stack.service       # expect:
        enabled
```

(iii) For BOB-137's owner (the file is theirs, `download-proxy/src/main.py`): the stall dumper's use of `faulthandler.dump_traceback(all_threads=True)` on a live multithreaded process is the crash vector. The upstream fix direction is *stop-the-world*; on 3.12 that is unavailable. Options are theirs to weigh — dump only the current thread, gate `all_threads` behind an opt-in, cap dumps per episode, or move the image to 3.13+. **No change was made here.**

Reproduction (not attempted on the live stack)

Per the task's constraint and §11.4.119, reproduction was **not** attempted: it requires either crashing the operator's running proxy or restarting the stack, and the stack is live with the operator on the machine. The procedure is handed over in §6(i). Verdict **(a)** rests on the causal chain in §3, which is established from primary evidence (byte-matched machine code + the container's own truncated dump + the upstream issue), not from a reproduction.

What was NOT the cause — the three lookalikes

Every one of these was checked and excluded from evidence. The reusable version of this is `docs/guides/container-death-triage.md`.

Lookalike	Verdict	Evidence
cgroup OOM-kill	**EXCLUDED**	`memory.events`: `oom_kill 0`, `oom_group_kill 0`. **Zero** `oom-kill` / `Memory cgroup out of memory` / `Killed process` lines in the entire 7.5-day journal retention. Flight recorder: `k_oom=0` and `oom_cgroups=""` across all 62 samples.
cgroup memory-ceiling pressure	**REAL, but not the cause**	`memory.max = 805306368` (768 MiB, matching `mem_limit: 768m`), `memory.peak = 805371904`, `memory.events: max 1584` in 48 min — the container lives *at* its ceiling. But `max` is **reclaim**, not a kill: `oom_kill` stayed 0. Containment working as designed. It cannot produce a SIGSEGV.
§12.12 thread / RLIMIT_NPROC exhaustion	**EXCLUDED**	`ulimit -u` = **65536** soft *and* hard; observed peak `unit_pids_peak = 1559` (**2.4 %**). No `EAGAIN`, no “failed to create new OS thread”, no `pthread_create failed`, no “Cannot

Lookalike	Verdict	Evidence
		allocate memory” anywhere. Distinct signature, absent.
Host memory pressure	**EXCLUDED**	Flight recorder (62 samples): `psi_mem_full10` max **0.64**, `mem_avail_kb` min **43.9 GB** of 62.6 GB.
Host power event during the SIGSEGV	**EXCLUDED**	`k_suspend=0`, `boot_changed=0`, no boot boundary near `2026-08-20 17:56`. (The *separate* 2026-08-19 event **was** a power-off — §4.)
conmon crash	**EXCLUDED**	§1.

> The flight recorder (`scripts/flight-recorder/`) began sampling at **2026-08-21T15:52:01Z**, *after* both events. It cannot retro-cover them; it is cited above only for the **current** baseline. Stated explicitly per §11.4.6 rather than presented as incident-time evidence.

Environment (measured, not assumed)

```
podman 5.7.1        conmon 2.2.1 (commit: alt1)      crun 1.27
rootless=true       cgroupVersion=v2
cgroupManager=systemd
EventLogger=journald   LogDriver=journald
OCIRuntime=crun
host python /usr/bin/python3 -> 3.14.6 (ALT Sisyphus)
container python /usr/local/bin/python3.12 -> 3.12.13
host TZ Europe/Belgrade (CEST, +0200);  journal retention from
2026-08-14T05:20:28+02:00
```

Timezone caution for anyone re-reading these logs: `podman ps` and some `podman events` records for the boba containers print **`+0300`** while the host journal prints **`+0200 CEST`**. The two disagree by an hour on the same instant. Every timestamp in this document is stated with its zone; the SIGSEGV is `2026-08-20 17:56:26 CEST` = `2026-08-20T15:56:26Z`.

Discovery-channel note (§11.4.238)

Both events were found by an agent investigating something else (BOB-129), not by container health monitoring — the escape BOB-131 itself records. The detector added by this investigation is `scripts/diagnostics/bob131_container_death_triage.sh`, which classifies both of these events correctly from raw evidence and distinguishes them from all three lookalikes. It is a **triage** detector, not a monitor: it answers *“what killed this container”*, not *“is the container up”*.